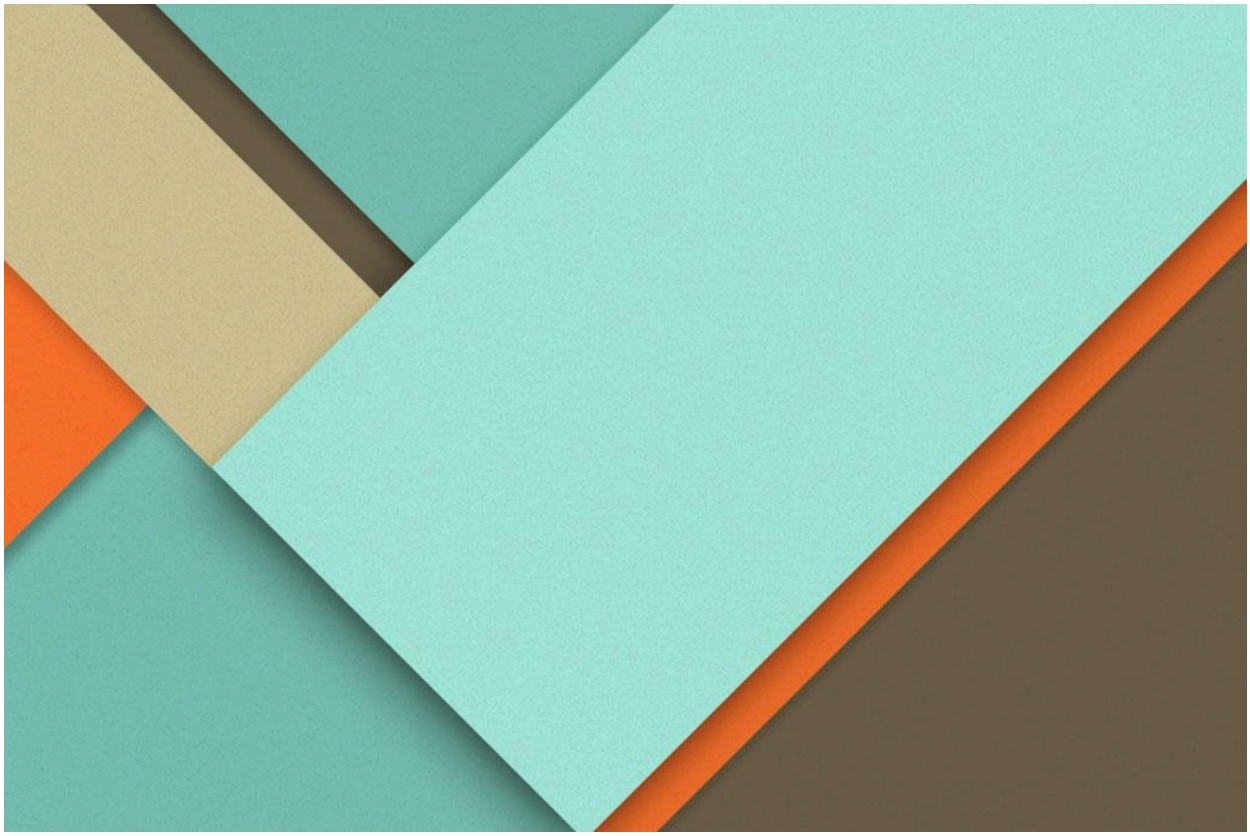


TP 4 Deep Learning

EL HADDAD Nassim

Exercice 1 - Exploration et prétraitement des partitions ABC

Le corpus contient 214 122 partitions pour l'entraînement et 2 162 pour la validation, toutes au format ABC. Le vocabulaire compte 95 symboles, avec des partitions de 290 caractères en moyenne et jusqu'à 2 968 caractères. Chaque partition commence par quelques métadonnées : `X:` pour l'identifiant, `L:` pour la durée de référence, `M:` pour la métrique et `K:` pour la tonalité. Viennent ensuite les notes et les barres de mesure.

Pourquoi trier le vocabulaire ?

Le vocabulaire est construit à partir d'un `set` de caractères. Comme son ordre n'est pas garanti, les indices pourraient changer d'une exécution à l'autre. On utilise donc `sorted(...)` pour garder une correspondance stable entre chaque caractère et son indice, aussi bien pendant l'entraînement que lors de la génération.

Pourquoi tronquer les partitions à 500 caractères ?

Les partitions font en moyenne 290 caractères. Fixer une longueur de 500 permet donc de garder l'essentiel de l'information sans devoir traiter les partitions les plus longues, qui peuvent atteindre 2 968 caractères. Utiliser directement cette longueur maximale serait beaucoup plus coûteux en mémoire GPU et en temps de calcul. Cela ajouterait aussi beaucoup de padding aux partitions courtes, donc des calculs inutiles.

Exercice 2 - Dataset auto-régressif et modèles

Avec `SEQ_LENGTH=500`, chaque partition est ramenée à 500 caractères, soit en la tronquant, soit en ajoutant du padding. Pour l'apprentissage auto-régressif, on décale ensuite la séquence d'un caractère : `x` contient les 499 premiers caractères et `y` les 499 suivants. Avec un batch de 128, les tenseurs ont donc une forme (128, 499). Les données sont en `torch.long` car elles correspondent aux indices utilisés par `nn.Embedding` et `CrossEntropyLoss`.

Nombre de paramètres :

Avec V = 95, E = 256 et H = 512, l'embedding compte 24 320 paramètres. Le RNN en compte 394 240 et la couche finale 48 735, soit 467 295 paramètres au total. Le LSTM possède quatre fois plus de paramètres dans sa partie récurrente, car il utilise quatre portes. Il compte ainsi 1 650 015 paramètres au total.

Architecture	Embedding	Bloc récurrent	FC	Total
RNN	24 320	394 240	48 735	467 295
LSTM	24 320	1 576 960	48 735	1 650 015

`nn.Embedding` est bien adapté ici car les caractères sont des catégories et non des valeurs numériques. Une couche linéaire pourrait laisser penser qu'il existe une distance entre les symboles, ce qui n'a pas vraiment de sens. L'embedding permet plutôt d'associer à chaque caractère un vecteur appris.

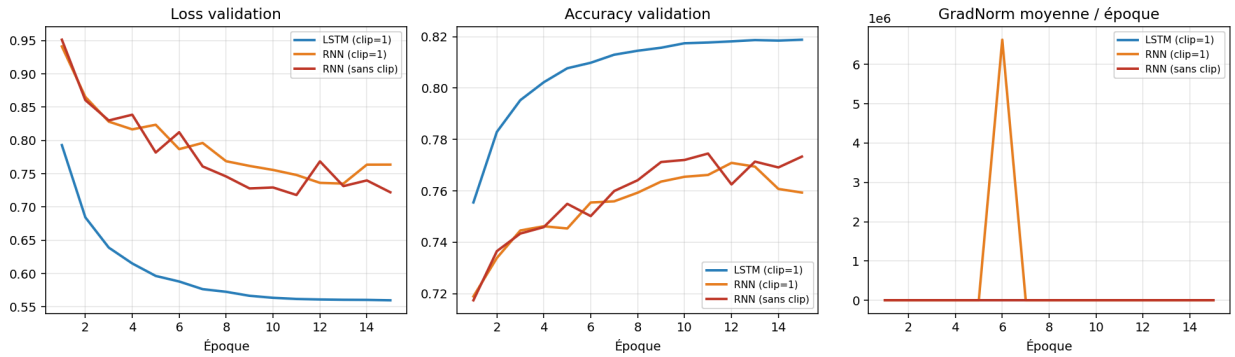
La principale différence entre les deux modèles concerne aussi leur mémoire interne. Le RNN utilise uniquement un état caché `h`, tandis que le LSTM utilise `h` et `c`. Ce dernier sert à conserver des informations sur une plus longue séquence grâce aux différentes portes du LSTM.

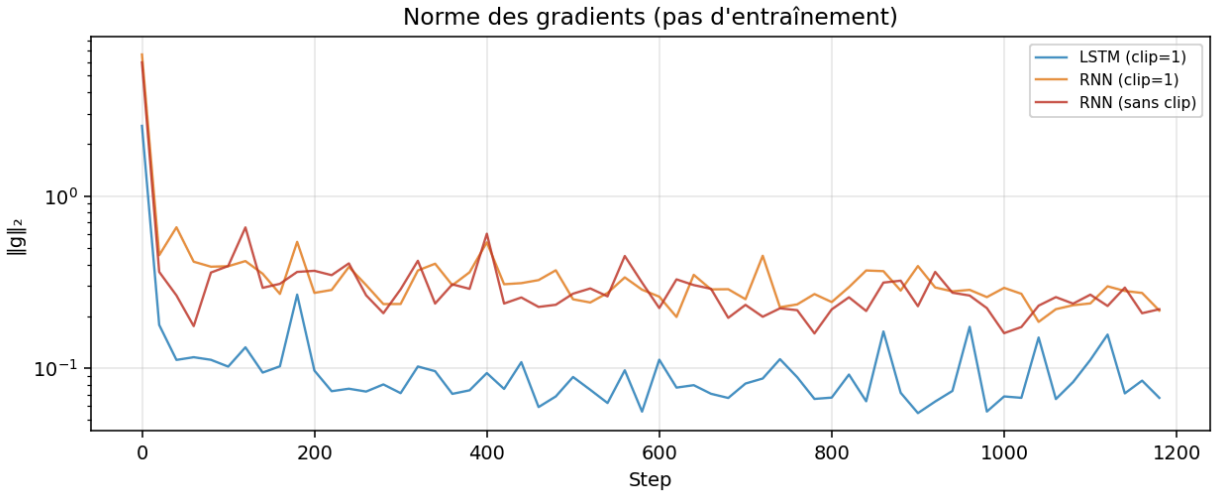
Exercice 3 - Entraînement et comparaison RNN / LSTM

Avant de calculer la `CrossEntropyLoss`, les tenseurs doivent être remis au bon format : les logits passent de (B, T, V) à (B × T, V) et les cibles de (B, T) à (B × T). Avec B = 128 et T = 499, cela représente 63 872 prédictions par batch. Le clipping des gradients est appliqué après `backward()` et avant `optimizer.step()`, afin de limiter leur norme avant la mise à jour des poids.

Sur le subset de 10 000 partitions, le LSTM obtient les meilleurs résultats, avec une Val Loss de 0,56 et une accuracy de 81,9 %, contre 77,3 % pour le RNN sans clipping et 75,9 % avec clipping.

Modèle	Val Loss	Val Acc	GradNorm moyenne
RNN sans clip	0.72	77.3%	0.22
RNN clip = 1	0.76	75.9%	0.27
LSTM clip = 1	0.56	81.9%	0.08





Le clipping est surtout utile pour le RNN. Vers la sixième époque, la norme des gradients atteint plusieurs millions, ce qui montre une explosion des gradients sur les longues séquences.

`clip_grad_norm_` limite cette norme avant la mise à jour des poids et rend l'apprentissage plus stable. En revanche, il ne règle pas le problème inverse : la disparition des gradients. Le LSTM gère mieux les longues séquences grâce à son *cell state* :

$$\mathbf{c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t}$$

Cette mise à jour permet au gradient de mieux circuler et au modèle de conserver des informations sur plusieurs pas, comme un rythme ou un motif musical. L"accuracy doit être interprétée avec prudence. Les partitions courtes sont complétées avec du padding, souvent facile à prédire, ce qui peut légèrement gonfler la métrique. On pourrait utiliser `CrossEntropyLoss(ignore_index=...)` pour ignorer ces caractères.

Exercice 4 - Génération et entraînement long

Complexité et température

Lors de la génération, le modèle conserve son état caché et ne traite qu'un nouveau caractère à chaque étape. Pour générer N caractères, la complexité est donc en O(N). Sans réutiliser l'état caché, il faudrait repasser toute la séquence à chaque étape, ce qui donnerait $O(N^2)$. La température T contrôle le niveau d'aléatoire. Avec une faible température, comme T = 0,2, la génération est plus prévisible mais aussi plus répétitive. Une température élevée apporte plus de diversité, mais peut rendre les résultats moins cohérents.

Quand T tend vers 0, la distribution se concentre sur le caractère le plus probable, ce qui se rapproche d'un décodage *greedy* (`argmax`). À l'inverse, quand T tend vers l'infini, les caractères deviennent presque équiprobables et la génération devient beaucoup plus aléatoire.

Exemples

Les générations à partir de l'amorce *Irish Reel* montrent bien l'effet de la température. À T = 0,2, la syntaxe ABC reste correcte, mais la sortie est très répétitive car le modèle choisit surtout les caractères les plus probables.

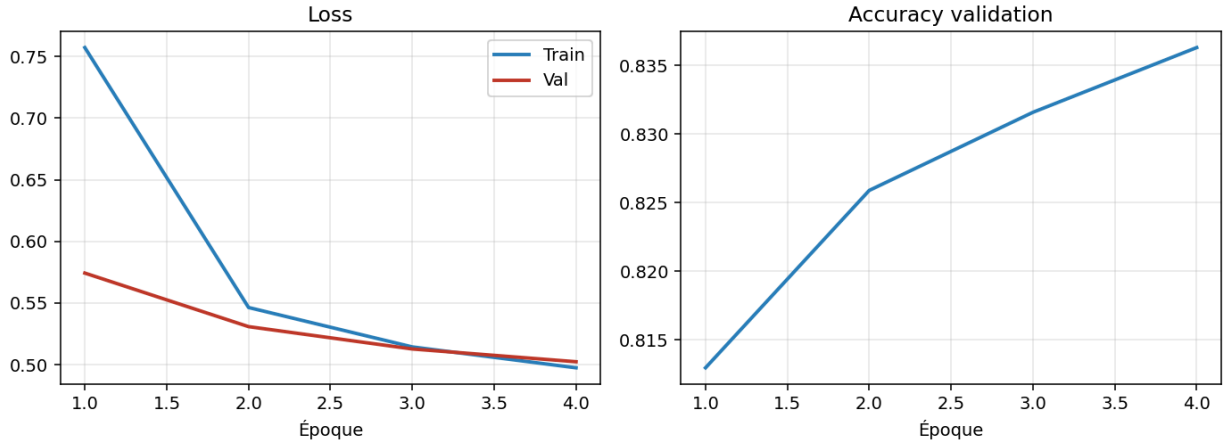
À T = 0,8, on obtient un bon compromis entre diversité et cohérence. Les notes et les rythmes sont plus variés tout en gardant une structure correcte.

À T = 2,0, la génération devient beaucoup plus aléatoire, avec davantage de caractères rares et d'erreurs de syntaxe. Une température trop élevée apporte donc plus de diversité, mais au détriment de la qualité.

Entraînement long avec Slurm

Un entraînement plus long a ensuite été réalisé avec un LSTM de taille cachée H = 1024, sur 50 000 partitions et jusqu'à 30 époques, avec un clipping des gradients à 1. Les résultats progressent rapidement dès les premières époques. À la sixième, la validation atteint 83,9 % d'accuracy avec une Val Loss de 0,50, déjà meilleure que celle obtenue avec le modèle entraîné sur 10 000 partitions.

Epoques	Train Loss	Val Loss	Val Acc
1	0.76	0.57	81.3%
2	0.55	0.53	82.6%
3	0.51	0.51	83.2%
4	0.50	0.50	83.6%
5	0.48	0.49	83.8%
6	0.48	0.50	83.9%



Avec plus de données, le modèle dispose de davantage d’exemples pour apprendre et peut mieux généraliser la structure musicale. Ces résultats montrent donc l’intérêt d’utiliser un corpus plus important pour cette tâche.

Transposition

Une piste simple pour augmenter la diversité du corpus serait de transposer les mélodies d'un certain nombre de demi-tons. Les notes ABC seraient alors décalées et la tonalité indiquée dans l'en-tête `K:` serait adaptée en conséquence. On garderait la structure rythmique et mélodique, tout en montrant la même idée musicale dans plusieurs tonalités. Le modèle dépendrait moins des tonalités dominantes du corpus, ce qui limiterait le sur-apprentissage.